Taller de pruebas unitarias en Java

JUnit 5 · Mockito · JaCoCo — Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

27 de octubre de 2026

1. Resultado

73 pruebas, todas en verde, y 98 % de cobertura de instrucciones con 100 % de ramas. Por el camino aparecieron cuatro errores en el código entregado: uno plantado en el ejercicio 1 y tres en el ejercicio 5.

```
$ mvn clean test

  Tests run:  1  ej0.SaludoTest
  Tests run: 10  ej1.CalculadoraNotasTest
  Tests run: 21  ej2.ValidadorContrasenaTest
  Tests run: 11  ej3.ServicioEstudiantesTest
  Tests run: 10  ej4.CarritoComprasTest
  Tests run: 20  ej5.EstadisticasTest
  ──────────────
  Tests run: 73, Failures: 0, Errors: 0, Skipped: 0      BUILD SUCCESS
```

Lo que más me llevo del taller no es el número sino un patrón: **los cuatro bugs estaban en los bordes**, ninguno en el caso normal. El valor límite exacto, el arreglo con todos los números negativos, la división de dos enteros, el último elemento de un recorrido. Si uno solo prueba “el caso que se supone que pasa”, los cuatro habrían quedado vivos.

2. Ejercicio 1 — Patrón AAA y el bug del valor límite

`CalculadoraNotas` tenía un error plantado. La regla del negocio dice que se aprueba con **mayor o igual a 3.0**, pero el código comparaba así:

```
public boolean aprobo(double nota) {
    return nota > NOTA_APROBATORIA;   // debería ser >=
}
```

Las pruebas de 4.5 y de 2.9 pasaban sin problema: una está claramente arriba y la otra claramente abajo. El error solo se ve en el punto exacto:

```
$ mvn test -Dtest=CalculadoraNotasTest

org.opentest4j.AssertionFailedError: 3.0 es el límite inclusive: debe aprobar
    ==> expected: <true> but was: <false>

Tests run: 10, Failures: 1
```

Tras cambiar > por >=, las 10 en verde. Agregué además los dos vecinos del límite, 2.99 y 3.01, porque dejan escrito en el código dónde está exactamente la frontera: si alguien la mueve, una prueba se queja.

Las tres partes del patrón quedan separadas a propósito:

```
@Test
@DisplayName("Valor límite: una nota de 3.0 exacta es aprobatoria")
void notaTresExactaAprueba() {
    double nota = CalculadoraNotas.NOTA_APROBATORIA;   // Arrange
    boolean resultado = calculadora.aprobo(nota);      // Act
    assertTrue(resultado, "3.0 es el límite inclusive: debe aprobar");  // Assert
}
```

También probé los extremos del rango válido, 0.0 y 5.0, que sí deben aceptarse, y las excepciones de lista vacía, lista nula y nota fuera de rango. En esta última no basta con verificar que lance: verifico el mensaje, porque es lo que de verdad le llega a quien usa la clase.

3. Ejercicio 2 — Excepciones y pruebas parametrizadas

21 pruebas sobre `ValidadorContrasena`. Una por cada regla incumplida, comprobando el mensaje exacto:

```
IllegalArgumentException ex = assertThrows(
        IllegalArgumentException.class,
        () -> validador.validar("segura2026")
);
assertEquals(ValidadorContrasena.MSG_MAYUSCULA, ex.getMessage());
```

Las parametrizadas evitan repetir la misma prueba cinco veces. Con `@ValueSource` agrupo contraseñas que deben fallar, y con `@CsvSource` empareja cada una con su mensaje:

```
@ParameterizedTest(name = "\"{0}\" debe fallar con: {1}")
@CsvSource({
        "Ab1,          Debe tener al menos 8 caracteres",
        "segura2026,   Debe tener al menos una mayúscula",
        "SeguraUIS,    Debe tener al menos un número"
})
void cadaContrasenaInvalidaTieneSuMensaje(String contrasena, String mensajeEsperado) { ... }
```

Dos casos que agregué por mi cuenta. El primero: una contraseña que incumple tres reglas a la vez (`"ab"` es corta, sin mayúscula y sin número) para dejar documentado que se reporta **la primera**; es un comportamiento que depende del orden de los `if` y que nadie había fijado. El segundo: el límite de longitud con 7 y 8 caracteres, otra vez el borde.

4. Ejercicio 3 — Aislar con Mockito

La primera prueba de este ejercicio no prueba la lógica, prueba **por qué hacen falta los mocks**:

```
@Test
@DisplayName("Con el repositorio real haría falta la base de datos institucional")
void conElRepositorioRealNoSePuedeProbar() {
    ServicioEstudiantes servicioReal = new ServicioEstudiantes(new RepositorioEstudiantesBD());

    UnsupportedOperationException ex = assertThrows(
            UnsupportedOperationException.class,
            () -> servicioReal.consultarEstado("2201234")
    );
    assertTrue(ex.getMessage().contains("base de datos"));
}
```

Con la implementación de verdad no se puede probar nada sin una conexión. El mock sustituye esa dependencia por una que responde lo que yo le diga:

```
when(repositorio.buscarPorCodigo("2201234")).thenReturn(Optional.of(ana));
```

Lo que hace posible el reemplazo es que `ServicioEstudiantes` recibe el repositorio **por el constructor**. Si lo construyera internamente, no habría por dónde meter el doble. Es la misma idea de los puertos del Lab 03: depender de una interfaz y no de una implementación concreta.

En `registrar()` lo que se comprueba cambia de naturaleza. El método no devuelve nada, así que no hay valor que comparar; lo que importa es **si la colaboración ocurrió o no**:

```
// con código válido: se llamó a guardar exactamente una vez
verify(repositorio, times(1)).guardar(nuevo);

// con código vacío: saltó la excepción y el repositorio quedó intacto
assertThrows(IllegalArgumentException.class, () -> servicio.registrar(invalido));
verifyNoInteractions(repositorio);
```

Esa segunda prueba es la que de verdad importa, porque verifica algo que es fácil romper sin darse cuenta: que la validación va **antes** de guardar y no después.

5. Ejercicio 4 — Carrito de compras con TDD

Cinco ciclos escribiendo primero la prueba. Lo que registré de cada rojo:

Ciclo	Qué agregué	El rojo que vi
1	Carrito nuevo vacío	`cannot find symbol: method cantidadProductos()`
2	Agregar un producto	`expected: <10000> but was: <0>`
3	Varios productos	pasó de una vez
4	Datos inválidos	`Expected IllegalArgumentException to be thrown, but nothing was thrown`
5	Descuento del 10 %	`expected: <108000> but was: <120000>`

Los rojos de los ciclos 1 y 2 son errores de compilación, no de aserción. Al principio desconcierta, pero es correcto: el método todavía no existe, y ese fallo **es** el rojo.

El ciclo 3 pasó sin escribir código nuevo, porque la suma del ciclo 2 ya resolvía varios productos. No es tiempo perdido: la prueba deja ese comportamiento protegido para que un refactor futuro no lo rompa en silencio.

El refactor final fue la mejor demostración de para qué sirven las pruebas. Reemplacé los contadores sueltos por una lista de ítems y saqué los números mágicos a constantes:

```
public static final long UMBRAL_DESCUENTO = 100_000;
public static final int PORCENTAJE_DESCUENTO = 10;

private record Item(String nombre, long precioUnitario, int cantidad) {
    long subtotal() { return precioUnitario * cantidad; }
}
```

Cambié la estructura de datos completa sin tocar una sola prueba, y las 10 siguieron en verde. Sin esa red, el mismo cambio habría requerido revisar a mano que todo siguiera funcionando.

Tres casos que agregué sobre lo que pedía la guía: cantidad negativa, precio 0 —que **sí** es válido, puede haber un producto de regalo— y el vecino del límite con $100.001, para dejar claro que "supera $100.000" significa > y no >=.

6. Ejercicio 5 — Los tres bugs

20 pruebas que destaparon ocho fallos. Siete eran bugs reales; uno era mío.

```
EstadisticasTest.maximoDeValoresNegativos        expected: <-2>  but was: <0>
EstadisticasTest.maximoDeUnSoloDato              expected: <-7>  but was: <0>
EstadisticasTest.medianaDeCantidadPar            expected: <2.5> but was: <2.0>
EstadisticasTest.contarAprobadosConUnaSolaNota   expected: <1>   but was: <0>
EstadisticasTest.contarAprobadosCuentaLaUltima   expected: <1>   but was: <0>
EstadisticasTest.contarAprobadosEnElLimite       expected: <1>   but was: <0>
EstadisticasTest.contarAprobadosConMezcla        expected: <3>   but was: <2>
```

Bug 1 — `maximo()`, valor inicial que no es neutro.

```
int max = 0;   →   int max = datos[0];
```

Arrancar en 0 parece inofensivo, pero 0 no está en los datos. Con `{-5, -2, -8}` ningún valor supera a 0, el `if` nunca se cumple y la función devuelve un número que no existe en el arreglo. Es interesante que `minimo()`, escrito justo debajo, **sí** parte de `datos[0]`: el mismo programador acertó en un método y falló en el otro.

Bug 2 — `mediana()`, división entera.

```
return (copia[n/2 - 1] + copia[n/2]) / 2;     →     ... / 2.0;
```

Los dos centrales de `{1,2,3,4}` son 2 y 3. `(2 + 3) / 2` entre enteros da 2, no 2.5: Java descarta los decimales antes de que el `double` de retorno entre en juego. Dividir entre `2.0` fuerza la promoción a `double`. El método **declara** que devuelve `double`, y eso hace fácil no ver el problema.

Bug 3 — `contarAprobados()`, uno de menos.

```
for (int i = 0; i < notas.length - 1; i++)   →   for (int i = 0; i < notas.length; i++)
```

El −1 deja fuera la última nota. Con un solo elemento el ciclo no se ejecuta ni una vez y siempre devuelve 0. Es el error clásico de uno en uno, y se detecta con exactamente los dos casos que dice la guía: un solo elemento, y comprobar que el último cuenta.

El fallo que era mío. Escribí que la mediana de `{1, 3, 2}` era 3.0, cuando ordenado queda `{1, 2, 3}` y el central es 2.0. La prueba estaba mal y el código tenía razón. Corregí la prueba. Sirve de recordatorio de que **una prueba en rojo no siempre acusa al código**: lo primero es verificar que el valor esperado sea el correcto.

7. Ejercicio 6 — Cobertura

```
CLASE                          INSTR   RAMAS   LÍNEAS
------------------------------------------------------
ej0/Saludo                      100%     n/a      2/2
ej1/CalculadoraNotas            100%    100%    11/11
ej2/ValidadorContrasena         100%    100%    13/13
ej3/Estudiante                  100%     n/a      1/1
ej3/RepositorioEstudiantesBD     44%     n/a      2/4
ej3/ServicioEstudiantes         100%    100%    18/18
ej4/CarritoCompras              100%    100%    16/16
ej4/CarritoCompras.Item         100%     n/a      2/2
ej5/Estadisticas                100%    100%    30/30
------------------------------------------------------
TOTAL                            98%    100%
```

Toda la lógica de negocio quedó al 100 % en instrucciones y en ramas. `RepositorioEstudiantesBD` se queda en 44 % y está bien que así sea: es la implementación contra base de datos, que en pruebas unitarias no se usa. Lo único ejecutado de ella es la prueba que demuestra por qué hacen falta los mocks.

El reporte sirvió para lo que dice la guía: **encontrar lo que falta probar**. `promedioGeneral()` salía en amarillo porque ninguna prueba pasaba por la rama de lista vacía, y el `if` de `registrar()` también, porque solo probaba el código vacío y no el nulo. Las dos pruebas que faltaban salieron de mirar el reporte:

```
@Test
@DisplayName("Si no hay estudiantes, el promedio general es 0.0")
void promedioGeneralSinEstudiantesEsCero() {
    when(repositorio.listarTodos()).thenReturn(List.of());
    assertEquals(0.0, servicio.promedioGeneral(), 0.001);
}
```

8. Preguntas de repaso

¿Por qué `assertEquals` con `double` necesita un tercer parámetro?

Porque los `double` no representan exactamente los decimales. `0.1 + 0.2` no da `0.3` sino `0.30000000000000004`, así que comparar por igualdad estricta falla por diferencias que no importan. El tercer parámetro es la tolerancia: con `assertEquals(4.0, resultado, 0.001)` se acepta cualquier valor a menos de una milésima, que para notas y promedios es de sobra.

¿Por qué una prueba unitaria no debería conectarse a la base de datos real?

Por cuatro razones que se notaron en el ejercicio 3. Deja de ser unitaria: si falla, no se sabe si el error está en la lógica o en la base. Deja de ser repetible: el resultado depende de qué datos haya en ese momento. Se vuelve lenta: milisegundos pasan a ser segundos, y una suite lenta se deja de ejecutar. Y deja de ser autónoma: no corre sin la base arriba, con sus credenciales y su red.

¿Qué es un "falso positivo" en pruebas y cómo se evita?

Una prueba que pasa aunque el código esté mal, de modo que da una seguridad que no existe. El caso típico es una prueba que en realidad no verifica nada: un `assert` que compara algo consigo mismo, o un `assertThrows` sobre una llamada que nunca se ejecuta. Se evita **viendo fallar la prueba primero**: si nunca se ha visto en rojo, no hay forma de saber si comprueba algo. Por eso el ejercicio 1 pide romper la prueba a propósito antes de seguir, y por eso en TDD el rojo es obligatorio.

¿Qué diferencia hay entre un stub y un mock?

El stub **responde**; el mock **vigila**. Un stub existe para dar datos de entrada a la prueba: `when(repositorio.buscarPorCodigo` solo sirve para que el servicio tenga con qué trabajar. Un mock se usa para comprobar que una interacción ocurrió: `verify(repositorio, times(1)).guardar(nuevo)` no mira ningún valor de retorno, mira que la llamada se hizo. En Mockito el objeto es el mismo; lo que cambia es para qué se usa. La regla práctica: si el resultado se ve en lo que devuelve el método, basta un stub; si el único efecto es una llamada a otro objeto, hace falta verificar.

Si un proyecto tiene 100 % de cobertura, ¿significa que no tiene errores?

No. La cobertura dice qué líneas se **ejecutaron**, no si alguien comprobó el resultado. Una suite de pruebas sin una sola aserción puede dar 100 %. Y este taller lo demuestra al revés: después de corregirlos, `CalculadoraNotas` y `Estadisticas` están al 100 % en instrucciones y ramas, pero con los bugs dentro habrían dado exactamente el mismo porcentaje, porque las líneas con el error se ejecutaban igual. Lo que faltaba no era pasar por esas líneas, sino pasar con **los datos correctos**: una nota de 3.0 exacta, un arreglo de negativos, un único elemento. La cobertura es útil para encontrar lo que nadie probó, no para afirmar que lo probado está bien.

9. Conclusión

Si tuviera que quedarme con una sola idea del taller sería que **los bugs viven en los bordes**. Los cuatro errores que encontré se escondían en el mismo tipo de sitio: el valor límite exacto, el arreglo donde el valor inicial "neutro" no es neutro, la división de dos enteros, el último elemento de un recorrido. Ninguno se veía con datos normales, y los cuatro aparecieron apenas escribí la prueba del caso incómodo.

De ahí sale lo segundo: diseñar casos es el trabajo real, y escribirlos es lo fácil. Pensar en particiones y valores límite antes de teclear fue lo que hizo que las pruebas encontraran algo en vez de confirmar lo que ya funcionaba.

Lo tercero lo entendí en el refactor del ejercicio 4. Cambié la estructura interna del carrito por completo y supe al instante que no había roto nada. Esa es la diferencia práctica entre tener pruebas y no tenerlas: no es detectar errores de hoy, es poder cambiar el código mañana sin miedo.

Y el cierre me lo dio mi propio error en el ejercicio 5. Una prueba en rojo es una pregunta, no un veredicto: puede estar mal el código, pero también la expectativa. Lo primero es verificar cuál de las dos.